

Activity: Git Branching and GitHub

Objective: To introduce participants to basic Git commands and workflows by creating a simple website and managing its version control.

Note: Insure activities of day 1 are complete before starting activities of day 2.

Working with branches

Ensure Clean Working Tree:

- Instruction: Before creating a feature branch, ensure your working directory is clean. Run `git status` and make sure there are no uncommitted changes.
- If there are, either commit them or stash them using `git stash`.
- Explanation: This ensures you're branching from a clean state.
- Participant Action: Run `git status`. If necessary, commit or stash changes.

Create a New Feature Branch:

- Instruction: Create a new branch named `feature/add-contact-form` using the command `git checkout -b feature/add-contact-form`.
- Explanation: This creates a new branch named `feature/add-contact-form` and immediately switches to it.
- Participant Action: Run the command.

Add a Contact Form:

- Instruction: Modify `index.html` to add a simple contact form. Add the following code within the `<body>` tag, preferably after the existing content:

Sample Code

```
<h2>Contact Us</h2>
<form action="/submit-contact" method="post">
  <label for="name">Name:</label><br>
  <input type="text" id="name" name="name" required><br><br>

  <label for="email">Email:</label><br>
  <input type="email" id="email" name="email" required><br><br>

  <label for="message">Message:</label><br>
```

```
<textarea id="message" name="message" rows="4" cols="50"
required></textarea><br><br>

<input type="submit" value="Submit">
</form>
```

- Participant Action: Open `index.html` in a text editor and add the code.

Stage and Commit the Changes:

- Instruction: Run `git add index.html` followed by `git commit -m "Add contact form to index.html"`.
- Explanation: This stages the changes and creates a commit on the feature branch.
- Participant Action: Run the commands.

Rename the Feature Branch:

- Instruction: While still on the `feature/add-contact-form` branch, rename it to `feature/contact-form` using the command `git branch -m feature/contact-form`.
- Explanation: This renames the current branch.
- Participant Action: Run the command.

Switch to the `main` Branch:

- Instruction: Run `git checkout main`.
- Explanation: This switches to the `main` branch.
- Participant Action: Run the command.

Merge the Feature Branch:

- Instruction: Merge the `feature/contact-form` branch into `main` using the command `git merge feature/contact-form`.
- Explanation: This integrates the changes from the feature branch into the main branch.
- Participant Action: Run the command. Resolve any merge conflicts if they arise.

Check the Commit Log:

- Instruction: Run `git log --oneline --graph`.
- Explanation: This displays the commit history in a concise, graphical format.
- Participant Action: Run the command and observe the output. You should see the merge commit and the commits from the feature branch.

Check the Log for a Specific File:

- Instruction: Run `git log --oneline index.html`.
- Explanation: This displays the commit history specifically for the `index.html` file.
- Participant Action: Run the command and observe the output. You should see the commits that modified `index.html`.

Steps to introduce merge conflict and solve it:

Create a New Feature Branch:

- Instruction: Create a new branch named `feature/conflict-test` using the command `git checkout -b feature/conflict-test`.
- Explanation: This creates a new branch and switches to it.
- Participant Action: Run the command.

Modify `index.html` on the Feature Branch:

- Instruction: Modify `index.html` on the `feature/conflict-test` branch. Add or change a line of text within an existing paragraph or heading. (or simply `<title>` tag)
- Participant Action: Modify the `index.html` file.

Stage and Commit Changes on the Feature Branch:

- Instruction: Run `git add index.html` followed by `git commit -m "Modify index.html on feature branch"`.
- Explanation: This stages and commits the changes on the feature branch.
- Participant Action: Run the commands.

Switch to the `main` Branch:

- Instruction: Run `git checkout main`.
- Explanation: This switches back to the `main` branch.
- Participant Action: Run the command.

Modify `index.html` on the `main` Branch:

- Instruction: Modify `index.html` on the `main` branch. Modify the *same* line of text that you modified on the `feature/conflict-test` branch, but make a *different* change. (note if you have modified `<title>` at previous step modify the same here)
- Participant Action: Modify the `index.html` file.

Stage and Commit Changes on the `main` Branch:

- Instruction: Run `git add index.html` followed by `git commit -m "Modify index.html on main branch"`.
- Explanation: This stages and commits the changes on the main branch.
- Participant Action: Run the commands.

Attempt to Merge the Feature Branch:

- Instruction: Run `git merge feature/conflict-test`.
- Explanation: This will attempt to merge the feature branch into the main branch, which will result in a conflict.
- Participant Action: Run the command. You should see a message indicating a merge conflict.

Resolve the Merge Conflict:

- Instruction:
 - Open `index.html` in a text editor.
 - Look for the conflict markers (`<<<<<<, =====, >>>>>>`).
 - Manually edit the file to resolve the conflict. Choose the changes you want to keep, or combine them as needed.
 - Remove the conflict markers.
 - Save the file.
- Participant Action: Resolve the merge conflict in `index.html`.

Stage the Resolved File:

- Instruction: Run `git add index.html`.
- Explanation: This stages the resolved file.
- Participant Action: Run the command.

Commit the Merge:

- Instruction: Run `git commit -m "Resolve merge conflict in index.html"`.
- Explanation: This commits the merge, completing the conflict resolution.
- Participant Action: Run the command.

Check the Commit Log:

- Instruction: Run `git log --oneline --graph`.
- Explanation: This displays the commit history, including the merge commit and the conflict resolution.
- Participant Action: Run the command and observe the output.

Congratulation! You have resolved the conflict!!

Creating a Remote Repository and Linking It to Local

Create a New Repository on GitHub:

- Instruction:
 - Go to GitHub and log in or create an account if you don't have one.
 - Click the "+" button in the top right corner and select "New repository."
 - Choose a repository name (e.g., "my-website").
 - Make the repository public or private, as you prefer.
 - Do not initialize the repository with a README, .gitignore, or license (we'll push our existing local repository).
 - Click "Create repository."
- Explanation: This creates an empty remote repository on GitHub.
- Participant Action: Follow the steps on GitHub.

Get the Remote Repository URL:

- Instruction: On the GitHub repository page, copy the HTTPS or SSH URL of your repository.
- Participant Action: Copy the URL.

Link the Local Repository to the Remote:

- Instruction: In your terminal, navigate to your local "my-website" repository. Run the following command, replacing `<remote_url>` with the URL you copied from GitHub:
 - `git remote add origin <remote_url>`
- Explanation: This adds a remote repository named "origin" to your local repository.
- Participant Action: Run the command, replacing the URL.

Verify the Remote:

- Instruction: Run `git remote -v`.
- Explanation: This displays the remote repositories and their URLs.
- Participant Action: Run the command and verify that the "origin" remote is listed.

Push the Local Repository to the Remote:

- Instruction: Run the following command to push your local `main` branch to the remote repository:
 - `git push -u origin main`
- Explanation: This pushes the `main` branch to the "origin" remote and sets it as the upstream branch.
- Participant Action: Run the command. You may be prompted to enter your GitHub username and password or use an authentication token.

Verify the Push on GitHub:

- Instruction: Go to your GitHub repository page and refresh it.
- Explanation: You should see your local repository's files and commit history on GitHub.
- Participant Action: Verify the push on GitHub.

Cloning and Pushing from Another Working Directory

Navigate to a Different Directory:

- Instruction: Open a new terminal window or navigate to a different directory on your computer, outside of your original "my-website" directory.
- Participant Action: Navigate to a different directory.

Clone the Remote Repository:

- Instruction: Run the following command, replacing `<remote_url>` with the URL of your GitHub repository:
 - `git clone <remote_url> cloned-website`
- Explanation: This clones the remote repository to a new directory named "cloned-website."
- Participant Action: Run the command.

Navigate to the Cloned Repository:

- Instruction: Run `cd cloned-website`.
- Participant Action: Run the command.

Make Changes to `index.html` in the Cloned Repository:

- Instruction: Open `index.html` in a text editor and make a small change (e.g., add a new paragraph or modify existing text).
- Participant Action: Modify `index.html`.

Stage and Commit Changes in the Cloned Repository:

- Instruction: Run `git add index.html` followed by `git commit -m "Add paragraph from cloned repo"`.
- Explanation: This stages and commits the changes in the cloned repository.
- Participant Action: Run the commands.

Push Changes from the Cloned Repository:

- Instruction: Run `git push origin main`.
- Explanation: This pushes the changes from the `main` branch of the cloned repository to the remote repository.
- Participant Action: Run the command. You may be prompted to enter your GitHub credentials.

Verify Changes on GitHub:

- Instruction: Go to your GitHub repository page and refresh it.
- Explanation: You should see the changes you pushed from the cloned repository.
- Participant Action: Verify the changes on GitHub.

Pull Changes in the Original Repository:

- Instruction: In your original "my-website" directory, run `git pull origin main`.
- Explanation: This pulls the changes from the remote repository into your original local repository.
- Participant Action: Run the command.

Verify Changes in the Original Repository:

- Instruction: Open `index.html` in your original "my-website" directory and verify that the changes from the cloned repository are present.
- Participant Action: Verify the changes.